

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

LENGUAJES DE PROGRAMACIÓN 1

2do.Examen

(Primer Semestre 2024)

Indicaciones Generales:

- Duración: 3 horas.

NO SE PERMITE EL USO DE APUNTES DE CLASE, FOTOCOPIAS NI MATERIAL IMPRESO

- No se pueden emplear variables globales, estructuras. La clase (o el tipo de datos) string solo podrá utilizarse donde se le indique. Tampoco se podrán emplear las funciones malloc, realloc, strdup o strtok, igualmente no se puede emplear cualquier función contenida en las bibliotecas stdio.h, cstdio o similares y que puedan estar también definidas en otras bibliotecas. **SOLO SE PODRÁ HACER USO DE PLANTILLAS Y STL EN AQUELLAS PREGUNTAS QUE ASÍ LO SOLICITEN**, DE LO CONTRARIO SE ANULARÁ LA PREGUNTA.
- Deberá modular correctamente el proyecto en archivos independientes. LAS SOLUCIONES DEBERÁN DESARROLLARSE BAJO UN ERICTO DISEÑO DESCENDENTE. Cada método **NO** debe sobrepasar las **20** líneas de código aproximadamente (una línea de código no es una línea de texto, es una instrucción que termina con un punto y coma. **LAS COMAS DEFINEN UNA LISTA DE INSTRUCCIONES POR LO QUE, SI EN UNA LÍNEA COLOCA INSTRUCCIONES SEPARADAS POR COMAS, SE CONTARÁ POR DOS CADA INSTRUCCIÓN**).
- El archivo **main.cpp** solo podrá contener la función **main** de cada proyecto y el **código contenido en él solo podrá estar conformado por tareas implementadas como métodos de clases**. En el archivo main.cpp o en el archivo que contenga el método main, deberá colocar un comentario en el que coloque claramente su nombre y código, **de no hacerlo se le descontará 0.5 puntos en la nota final. (NO SE HARÁN EXCEPCIONES)**.
- El código comentado **NO SE CALIFICARÁ y esto incluye el comentar el llamado a la función que lo contiene**.
- La cláusula **friend** solo se podrá emplear en el caso de clases auto referenciadas para ligar el nodo con la clase **inmediata** que encapsula la lista, **en ningún caso adicional**. No se considerará en la nota las clases que violen esto. Tampoco se podrá emplear la cláusula **protected**.
- Deberá mantener en todo momento el encapsulamiento de todos los atributos de las clases, así como guardar los estándares en la definición y uso de todas las clases desarrolladas.
- Salvo en la sobrecarga de los operadores >> y <<, no se podrán definir funciones (ni plantillas de funciones) independientes que no estén ligadas como métodos a alguna de las clases planteadas.
- **Todos los métodos y funciones ligados a una clase deben estar desarrollados en el mismo archivo.**
- **NO PUEDE EMPLEAR ARREGLOS NI ARCHIVOS AUXILIARES PARA COLOCAR PARTE O TODOS LOS DATOS DE LOS ARCHIVOS DADOS**
- Los programas que presenten errores de sintaxis o de concepto se calificarán en base al 40% de puntaje de la pregunta. Los que no den resultados coherentes en base al 60%.
- Se tomará en cuenta en la calificación el uso de comentarios relevantes.
- **NO PUEDE UTILIZAR VARIABLES ESTÁTICAS.**

SE LES RECUERDA QUE, DE ACUERDO AL REGLAMENTO DISCIPLINARIO DE NUESTRA INSTITUCIÓN, CONSTITUYE UNA FALTA GRAVE COPIAR DEL TRABAJO REALIZADO POR OTRA PERSONA O COMETER PLAGIO.

NO SE HARÁN EXCEPCIONES ANTE CUALQUIER TRASGRESIÓN DE LAS INDICACIONES DADAS EN LA PRUEBA

Puntaje total: 20 puntos

INDICACIONES INICIALES

- La unidad de trabajo será **t:** (Si lo coloca en otra unidad, no se calificará su laboratorio y se le asignará como nota cero)
- Cree allí una carpeta con el nombre **"EX_FINAL_CO_PA_PN_2024_1"** donde **CO** indica: Código del alumno, **PA** indica: Primer Apellido del alumno y **PN** primer nombre (de no colocar este requerimiento se le descontará 3 puntos de la nota final). **Allí colocará los proyectos solicitados en la prueba.**

PREGUNTA 1 (7 puntos)

Se solicita que desarrolle un proyecto "PasaLibros" dentro de la carpeta correspondiente, **DE NO COLOCAR ESTE REQUERIMIENTO SE LE DESCONTARÁ 2 PUNTOS DE LA NOTA FINAL**, en la cual se declaren las clases descritas con las relaciones necesarias, que permitan manipularlas empleando herencia:

- **Para manejar los libros:** La clase se denominará "Libro" y deberá contener lo siguiente: 1) un atributo denominado **nombre** (**char***) definido por una cadena dinámica de caracteres que representa el título del libro, 2) un atributo denominado **paginas** (**int**) que representa la cantidad de páginas que tiene el libro, 3) un atributo denominado **peso** (**double**) que representa el peso del libro.
- **Para manejar los libros tipo novela:** La clase se denominará "Novela" y deberá contener lo siguiente: 1) un atributo denominado **autor** (**char***) definido por una cadena dinámica de caracteres que representa el nombre del autor del libro, 2) un atributo denominado **editorial** (**char***) definido por una cadena dinámica de caracteres que representa el nombre la editorial del libro. Esta clase posee datos heredados de la clase **Libro**.
- **Para manejar los libros tipo revista:** La clase se denominará "Revista" y deberá contener lo siguiente: 1) un atributo denominado **ISSN** (**int**) que representa el código ISSN de la publicación, 2) un atributo denominado **anho** (**int**) que representa el año que fue publicada la revista, 3) un atributo denominado **numero** (**int**) que representa de la revista. Esta clase posee datos heredados de la clase **Libro**.
- **Para manejar los ejemplares:** La clase se denominará "Ejemplar" y deberá contener lo siguiente: 1) un atributo denominado **plibro**, este es un puntero de la clase **Libro**, 2) un atributo denominado **sig**, este atributo es un puntero a la clase **Ejemplar** (autoreferenciado).
- **Para manejar la Pila de ejemplares:** La clase se denominará "Pila" y deberá contener lo siguiente: 1) un atributo denominado **nelementos** (**int**) que representa la cantidad ejemplares que tiene almacenada la pila, 2) un atributo denominado **pcima**, este atributo es un puntero de clase **Ejemplar**.
- **Para manejar el proceso:** La clase se denominará "Procesa" y deberá contener lo siguiente: 1) un atributo denominado **porigen** dado por un objeto de la clase **Pila**, 2) un atributo denominado **pdestino** dado por un objeto de la clase **Pila**.

"DEBE EMPLEAR OBLIGATORIAMENTE LOS NOMBRES DE LAS CLASES Y SUS ATRIBUTOS"

Consideraciones:

Para el desarrollo de la pregunta debe considerar el siguiente código:

```
#include "Procesa.h"

using namespace std;

int main(int argc, char** argv) {
    Procesa pro;

    pro.carga();
    pro.pasa();
    pro.muestra();

    return 0;
}
```

**NO PUEDE
CAMBIAR
ESTE CÓDIGO**

Con las clases indicas debe realizar las siguientes operaciones:

- **(3 puntos)** En la clase **Procesa** implementar el método **carga**, que se encarga de la lectura del archivo "Libros5.csv" y colocará el libro leído en el objeto **Ejemplar**, para la lectura de los datos correspondiente de cada libro debe utilizar el método polimórfico **lee**, ya que los mismos pueden ser novelas o revistas. Para diferenciar cada tipo de libro en el archivo correspondiente las novelas se representan con la letra "N" y las revistas con la "R". Con el objeto **Ejemplar** cargado debe colocar el elemento en la Pila **porigen** incrementando el atributo **nelementos**. Para esta pregunta debe implementar los métodos necesarios que utiliza una pila, como son: **apilar**, **desapilar** y **pilavacia**. Los métodos no deben recibir o devolver punteros, excepto las que manejan cadenas de caracteres.

- (3 puntos) En la clase **Procesa** implementar el método **pasa**, que se encargará de mover los ejemplares de la pila **porigen** a la pila **pdestino**, este método debe colocar los libros en la pila destino en el mismo orden que tenían en la pila original.
- (1 punto) En la clase **Procesa** implementar el método **muestra**, que se encarga de realizar la impresión de un archivo debidamente tabulado (**sin usar el carácter '\t'**), que muestre los datos de cada **Ejemplar**. Recuerde que es una pila por tal motivo no se puede recorrer en ningún momento. Para la impresión de los libros contenidos en cada **Ejemplar** debe usar el método polimórfico **imprime**. Esta pregunta es válida solo se realizó el método carga de forma correcta. El reporte debe tener el siguiente formato:

```
Reporte de Ejemplares
=====
Titulo:El nombre de la rosa
Peso: 0.60
Autor: Umberto Eco
Editorial: Planeta

Titulo:Boletin de Arqueologia PUCP
Peso: 0.15
ISSN:10292004    Año: 2020

Titulo:Don Quijote de la Mancha
Peso: 1.00
Autor: Miguel de Cervantes
Editorial: Salvat
...
```

Se recomienda revisar los archivos que servirán para la lectura de datos, los cuales se describen a continuación:

```
Libros5.csv
N,El nombre de la rosa,105,0.6,Umberto Eco,Planeta
R,Boletin de Arqueologia PUCP,48,0.15,10292004,2020,3
N,Don Quijote de la Mancha,120,1,Miguel de Cervantes,Salvat
...
```

Tipo, Titulo, Páginas, Peso, Autor/ISSN, Editorial/Año, Número

Recuerde que si no usa polimorfismo la respuesta no será válida, así mismo siempre debe mantener el encapsulamiento por lo cual debe realizar cada operación donde le corresponde, por ejemplo, si va a leer o imprimir un libro esta operación la debe realizar en la clase que le corresponde. La cláusula *friend* solo se puede emplear para acceder desde la TAD al nodo (Ejemplar).

Pregunta 2 (7 puntos):

Se solicita que desarrolle un proyecto **"ExamenFinal_PREG02"** dentro de la carpeta correspondiente, **DE NO COLOCAR ESTE REQUERIMIENTO SE LE DESCOTARÁ 2 PUNTOS DE LA NOTA FINAL**, en la cual se declaren las clases descritas con las relaciones necesarias, que permitan manipularlas empleando los conocimientos vistos en clas:

- **Para manejar los libros:** La clase se denominará **"Libro"** y deberá contener lo siguiente: 1) un atributo denominado **codigo** (**string**) definido por objeto de la clase string que representa el código del producto, 2) un atributo denominado **titulo** (**string**) que representa el título del libro, 3) un atributo denominado **autor** (**string**) que representa el código del autor del libro, 4) **genero**(**string**) que representa el género del libro, 5) **tema**(**string**) que representa el tema del libro.
- **Para manejar los autores:** La clase se denominará **"Autor"** y deberá contener lo siguiente: 1) un atributo denominado **codigo** (**string**) definido por objeto de la clase string que representa el código del autor, 2) un atributo denominado **nombre** (**string**) que representa el nombre del autor, 3) un atributo denominado **libros** este es un STL-List de la clase **Libro**, donde se guardarán los libros del autor.
- **Para manejar los clientes:** La clase se denominará **"Cliente"** y deberá contener lo siguiente: 1) un atributo denominado **codigo** (**string**) definido por objeto de la clase string que representa el código del cliente, 2) un atributo denominado **nombre** (**string**) que representa el nombre del autor, 3) un atributo denominado **libros_comprados** este es un STL-Vector de la clase **Libro**, 4) un atributo denominado **cantidad_libros_vendidos**(**int**), 5) un atributo denominado **recomendaciones** este es un STL-Map, donde la llave es un objeto de la clase **Libro** y el valor es un STL-Vector de la clase **Libro**.

- **Para manejar la biblioteca:** La clase se denominará **"Biblioteca"** y deberá contener lo siguiente: 1) un atributo denominado **autores** este será definido por STL-Map, donde la llave es un String que representa al código del autor y el valor es un objeto de la clase Autor, 2) un atributo denominado **libros** este será definido por STL-Vector de la clase Libro, 3) un atributo denominado **clientes** que será definido por un STL-Vector de la clase Cliente.

"DEBE EMPLEAR OBLIGATORIAMENTE LOS NOMBRES DE LAS CLASES Y SUS ATRIBUTOS"

Con las clases indicas debe realizar las siguientes operaciones:

- En la clase **Biblioteca**, implementar el método **cargar_libros**, que se encarga de la lectura del archivo **"Libros.csv"** y cargar la información en el STL-Vector denominado **libros**.
- En la clase **Biblioteca**, implementar el método **cargar_autores**, que se encarga de la lectura del archivo **"Autores.csv"** y cargar la información en el STL-Map denominado **autores**. Recuerde completar la STL-List **libros** con la ayuda del vector libros.
- En la clase **Biblioteca**, implementar el método **cargar_clientes**, que se encarga de la lectura del archivo **"Clientes.csv"** y carga la información del STL-Vector denominado **clientes**. Este metodo deberá encargarse también de calcular las recomendaciones de cada cliente y llenar la estructura **recomendaciones**. Siguiendo las siguientes consideraciones
 - Un libro es recomendado si comparte el mismo Autor.
 - Un libro es recomendado si son del mismo Género.
 - Un libro es recomendado si tratan el mismo Tema.
 - Recuerde que NO debe recomendar el mismo libro.
- En la clase **Biblioteca**, implementar un método que permita mostrar el siguiente reporte:

```

=====
REPORTE DE RECOMENDACIONES
=====
Codigo de Cliente: ABC1234 Nombre: Erasmo Gomez
Libro Comprado 1) El libro de la Selva
Libros Recomendados: El libro de la Sierra
                      El libro de la Felicidad
Libro Comprado 2) El libro de la Sierra
Libros Recomendados: El libro de la Selva
Libro Comprado 3) El libro de la Costa
Libros Recomendados: El libro de la Felicidad
Libro Comprado 4) El libro de la Felicidad
Libros Recomendados: El libro de la Selva
                      El libro de la Costa
=====
...

```

Consideraciones:

Para el desarrollo de esta pregunta considere el siguiente código:

```

#include "Biblioteca.hpp"

int main() {
    Biblioteca biblioteca;
    biblioteca.cargar_libros("Libros.csv");
    biblioteca.cargar_autores("Autores.csv");
    biblioteca.cargar_clientes("Clientes.csv");
    biblioteca.mostrar_recomendaciones("Reporte.txt");
    return 0;
}

```

**NO PUEDE
CAMBIAR
ESTE
CÓDIGO**

PREGUNTA 3 (6 puntos)

Crea una carpeta denominada **"Pregunta01_Programa_En_Java"**, dentro de la carpeta anteriormente descrita, y en ella desarrollará el programa que dé solución al problema planteado. **DE NO COLOCARSE ESTE REQUERIMIENTO SE LE DESCONTARÁ DOS (2) PUNTOS DE LA NOTA FINAL.**

EN ESTA PREGUNTA SOLO PODRÁ UTILIZAR EL NotePad++ PARA DESARROLLAR EL PROGRAMA. DE EMPLEAR OTRO ENTORNO SE LE DESCONTARÁ LA MITAD DEL PUNTAJE

CADA CLASE SE DEBE DESARROLLAR EN UN ARCHIVO INDEPENDIENTE .java, SI EN UN ARCHIVO .java COLOCA DOS O MÁS CLASES ÉSTAS NO SERÁN CALIFICADAS.

DEBERÁ MANTENER EN TODO MOMENTO EL ENCAPSULAMIENTO DE LOS ATRIBUTOS, POR LO QUE DEBE DECLARAR TODO ATRIBUTO COMO PRIVADO, SINO SE DESCONTARÁ 0.5 EN CADA ACASO.

Un restaurante maneja un archivo como el que se muestra a continuación:

```
R TER-1004 Terraza 2 María-Gómez
R SAL-1004 Salón 6 Ana-Martínez
P SP1-1001 Salón-Elite 10 Laura-Sánchez Alejandro-Díaz 3 Televisor Bar Audio
...
FIN
R 98765432 Sergio-Díaz T 2 21.5 N
C 21098765 Hugo-Sánchez T 2 19.5 A 2 Nueces Trigo
P 54101098 Hugo-Pérez 10 G Televisor Bar Audio 3
...
```

En la primera parte del archivo vienen los datos de las mesas del restaurante que están disponibles en una fecha determinada. En cada línea se ha colocado el tipo de mesa (R: Regular, P: Premium), el código de la mesa, la ubicación de la mesa (Terraza, Salon, Salon-Elite, Salón-Ejecutivo, Salon-VIP), el nombre del camarero(s) asignado(s) a la mesa y la capacidad (cantidad de personas) de la mesa. Para las mesas Premium también se ha colocado la cantidad y los nombres de las amenidades proporcionadas. La información de las mesas termina con la palabra FIN.

En la segunda parte se coloca la lista de solicitudes de reserva realizadas. En cada línea se ha colocado el tipo de solicitud (R: Regular, C: Con Restricciones, P: Premium), el DNI y el nombre de la persona que realizó la reserva, la ubicación deseada (T: Terraza, S: Salón), la cantidad de personas, la hora de la reserva y la ocasión (N: No Aplica, C: Cumpleaños, A: Aniversario, G: Graduación). Hay que tener en cuenta que el restaurante solo cuenta con dos turnos, el primer turno inicia a las 19:30 pm. y el segundo turno a las 21:30 pm., la hora de las solicitudes solo podrán tener los valores 19.5 o 21.5. En el caso de las solicitudes de reservas Premium no se coloca la ubicación deseada ni la hora ya que estas solicitudes serán asignadas a los salones privados (Salon-Elite, Salon-Ejecutivo o Salon-VIP) según disponibilidad y estas mesas son reservadas desde la apertura hasta el cierre del restaurante (único turno), la hora de estas solicitudes siempre será 19:30 pm., también se han colocado el nombre y la cantidad de las amenidades solicitadas.

Se desea elaborar una aplicación en lenguaje JAVA que permita procesar la información de este archivo, la aplicación definirá las clases que se describen a continuación.

- **Clase Restaurante:** la clase debe contener los siguientes atributos: 1) **mesas** (**ArrayList**) que contendrá las mesas disponibles en el restaurante 2) **solicitudesReservas** (**ArrayList**) que contendrá las solicitudes de reserva del restaurante.
- **Clase Mesa:** clase abstracta que debe contener los siguientes atributos: 1) **codigo** (**String**), 2) **ubicacion** (**String**), 3) **capacidad** (**int**), 4) **ocupadaPrimerTurno** (**boolean**) que especificará si la mesa ya se encuentra ocupada para el primer turno.
- **Clase MesaRegular:** que herede de la clase Mesa, debe contener los siguientes atributos: 1) **camarero** (**String**) que contendrá el nombre del camarero asignado a la mesa, 2) **ocupadaSegundoTurno** (**boolean**).
- **Clase MesaPremium:** que herede de la clase Mesa, debe contener los siguientes atributos: 1) **camareros** (**ArrayList**) que contendrá los nombres de los camareros asignados a la mesa, 2) **amenidades** (**ArrayList**) que contendrá la lista de amenidades que ofrece la mesa, 3) **numeroDeAmenidades** (**int**).
- **Clase SolicitudReserva:** clase abstracta que debe contener los siguientes atributos: 1) **dni** (**int**) que contendrá el DNI de la persona que realizó la reserva, 2) **nombre** (**String**) que contendrá el nombre de la persona que realizó la reserva, 3) **ubicacionDeseada** (**char**) contendrá la ubicación en el restaurante que prefiere el cliente (T: Terraza, S: Salon, E: Para solicitudes Premium), 4) **cantidadPersonas** (**int**) contendrá la cantidad de personas en la reserva, 5) **hora** (**double**) que contendrá la hora de la reserva, 6) **ocasion** (N: No Aplica, C: Cumpleaños, A: Aniversario, G: Graduación), 7) **mesaAsignada** (**Mesa**) que contendrá el código de la mesa que se asignó a la reserva, 8) **estado** (**char**) (R: Rechazada, A: Aprobada).
- **Clase SolicitudReservaRegular:** que herede de la clase SolicitudReserva, esta clase no cuenta con atributos adicionales, pero tener en cuenta que sobre escribe e implementa métodos polimórficos.

- **Clase SolicitudReservaConRestricciones:** que herede de la clase SolicitudReserva, debe contener los siguientes atributos: 1) **alergias** (**ArrayList**) que contendrá las restricciones alimentarias para tener en cuenta en la reserva, 2) **numeroDeAlergias** (**int**) número de restricciones alimentarias.
- **Clase SolicitudReservaPremium:** que herede de la clase SolicitudReserva, debe contener los siguientes atributos: 1) **amenidades** (**ArrayList**) que contendrá de lista de amenities que se requieren en la reserva, 2) **numeroDeAmenidades** (**int**). Las reservas Premium no cuentan con ubicación deseada dado que son asignadas a una mesa en un salón privado en este caso se usará E: Elite, así como para la hora de la reserva, una reserva Premium no tiene hora de llega ya que la mesa está disponible desde la hora de apertura hasta la hora de cierre del restaurante, se debe usar un valor de 19.5 (19:30 pm.).

"DEBE EMPLEAR OBLIGATORIAMENTE LOS NOMBRES DE LAS CLASES Y SUS ATRIBUTOS"
"NO PUEDE AGREGAR MÁS ATRIBUTOS A LAS CLASES"

La aplicación deberá leer a través de la clase Restaurante, las características de cada mesa, así como las características de cada solicitud de reserva. Se deberá asignar a cada reserva una mesa de acuerdo a las características de la reserva, una reserva regular deberá ser asignada a una mesa que se encuentre disponible en Terraza o Salón, teniendo en cuenta la capacidad de la mesa la cual no se puede exceder, en caso ya no queden mesas disponibles en la ubicación deseada en la solicitud de reserva se asignará una mesa en otra ubicación (terrazza o salón).

La asignación de mesas debe tener en cuenta la hora de la solicitud de reserva ya que para el caso de mesas regulares se manejan dos turnos (una mesa puede ser asignadas a dos solicitudes si estas se encuentran en diferentes horarios). En el caso de una reserva Premium se asignará una mesa ubicada en un salón privado teniendo en cuenta la capacidad de la mesa y la que cumpla con las amenities solicitadas.

En caso no se pueda encontrar una mesa para una reserva (porque ya no hay mesas disponibles) la mesa asignada en la solicitud de reserva tendrá un valor de null y el estado de la solicitud se colocará como R: Rechazado. Finalmente la aplicación deberá emitir un reporte también a través de la clase Restaurante donde se muestre muy claramente los datos de las solicitudes de reserva incluyendo la mesa que fue asignada, el camarero o camareros asignados, y las características de la reserva: dni, nombre, hora (en formato HH:MM pm.), ocasión (cumpleaños, aniversario, graduación), cantidad de personas, y para el caso de reservas con restricciones a lista de las restricciones y para el caso de reservas Premium la lista de amenities que fueron solicitadas.

Deberá definir por cada clase, los métodos selectores que se requieran y el constructor por defecto si se necesita. NO DEBE DEFINIR CONSTRUCTORES CON PARÁMETROS. También será obligatorio definir y utilizar métodos de lectura y escritura en cada clase que trabajen de manera polimórfica. **NO SE PODRÁ LEER O IMPRIMIR DATOS QUE PERTENEZCAN A OTRA CLASE.**

Al finalizar el examen, comprima la carpeta de su proyecto empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares.** Luego súbalo a la tarea programa en Paideia para este examen.

Profesor del curso:	Rony Cueva	Eric Huiza
	Erasmó Gómez	Heider Sánchez
	Miguel Guanira	

San Miguel, 9 de julio del 2024.